

GRAPH

PROBLEM 8

QUESTION:

DS	Session	Graph	Question Information	Level 1	Challenge 88
Question description You are given a tree consisting of n nodes. A <i>matching</i> is a set of edges where each node is an endpoint of at most one edge. What is the maximum number of edges in a matching? Input Format The first input line contains an integer n: the number of nodes. The nodes are numbered $1, 2, \dots, n$. Then there are $n-1$ lines describing the edges. Each line contains two integers a and b: there is an edge between nodes a and b. Output Format Print one integer: the maximum number of pairs. Constraints $1 \leq n \leq 2 \cdot 10^5$ $1 \leq a, b \leq n$ Example Input: <pre>5 1 2 1 3 3 4 3 5</pre> Output: <pre>2</pre> Explanation: One possible matching is $\{1, 2\}$ and $\{3, 4\}$					
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>5 1 2 1 3 3 4 3 5</pre> EXPECTED OUTPUT <pre>2</pre> Test Case 2 INPUT (STDIN) <pre>5 1 1 1 2 2 3 3 4</pre> EXPECTED OUTPUT <pre>6</pre>					
Mandatory Test Cases Test Case 1 KEYWORD <pre>void dfs(int p,int i)</pre> Test Case 2 KEYWORD <pre>void link(int i,int j)</pre>					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY <pre>9</pre> Test Case 2 TOKEN COUNT <pre>350</pre> Test Case 3 NLOC <pre>50</pre>					

SOURCE CODE:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_NODES 200005

int tree_head[MAX_NODES], edge_dest[2 * MAX_NODES], edge_next_ptr[2 * MAX_NODES], edge_ptr = 0;
bool node_paired[MAX_NODES];
int total_matched_pairs = 0;

void insert_tree_link(int u, int v) {
    edge_dest[++edge_ptr] = v;
    edge_next_ptr[edge_ptr] = tree_head[u];
    tree_head[u] = edge_ptr;

    edge_dest[++edge_ptr] = u;
    edge_next_ptr[edge_ptr] = tree_head[v];
    tree_head[v] = edge_ptr;
}

void solve_matching_dfs(int parent, int cur) {
    for (int e = tree_head[cur]; e; e = edge_next_ptr[e]) {
        int neighbor = edge_dest[e];
        if (neighbor != parent) {
            solve_matching_dfs(cur, neighbor);
            if (!node_paired[cur] && !node_paired[neighbor]) {
                node_paired[cur] = true;
                node_paired[neighbor] = true;
                total_matched_pairs++;
            }
        }
    }
}

int main() {
    int nodes;
    if (scanf("%d", &nodes) != 1) return 0;

    for (int i = 0; i < nodes - 1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        insert_tree_link(u, v);
    }

    solve_matching_dfs(0, 1);
    printf("%d\n", total_matched_pairs);
    return 0;
}
```

OUTPUT:

```
5
1 2
1 3
3 4
3 5
2
```

```
5
1 1
1 2
2 3
3 4
6
```